

Topic 30 — Introduction to Prompt Serialization Styles

Full Stack Python + AI Roadmap • Phase 4: Prompt Serialization & Instruction Formats • 3 layers: New → Intermediate → Expert



Layer 1 — New to the Concept

Serialization means converting something into a specific text format for storage or transmission. You know this already — `JSON.stringify()` serializes a JS object into a string; `json.dumps()` does it in Python.

Prompt serialization applies the same idea to prompts: converting your structured prompt (roles, messages, instructions) into the exact **plain-text string** a specific model was trained to understand.

The key realization: when you write this friendly structure —

```
messages = [
    {"role": "system", "content": "You are helpful."},
    {"role": "user", "content": "Hello!"}
]
```

— the model doesn't receive a list of dictionaries. Under the hood, that gets **serialized into one long text string** with special markers, and *that* string is what the model reads. Different models expect **different string formats** — the "serialization style."

i Analogy: the same address can be written in different formats for different countries' postal systems. Same information (recipient, street, city), but each service expects its own layout. Models are the same — same prompt content, each family wants its own text layout.



Layer 2 — What Serialization Actually Looks Like

When you send that clean `messages` list to an API, it gets flattened into a single string with **special tokens** marking each part:

```
# Conceptually, your messages become something like:
<|system|>
You are helpful.
<|user|>
Hello!
<|assistant|>
```

Those `<|system|>`, `<|user|>`, `<|assistant|>` markers are **special tokens** — they tell the model "a new role's turn starts here." The model was *trained* on text formatted exactly this way, so it recognizes the structure, knows whose turn is whose, and where to start generating (right after `<|assistant|>`).

Why different formats exist

Each model family was trained with its *own* markers and layout. Feed a model the wrong format and it works worse — the structure doesn't match what it learned. The three big formats in this phase:

```
Alpaca    → uses "### Instruction:", "### Response:" text headers
ChatML    → uses <|im_start|>role ... <|im_end|> markers (OpenAI)
INST      → uses [INST] ... [/INST] tags (LLaMA-2)
```

The crucial distinction — API vs raw model

```
# Hosted API (OpenAI, Anthropic):
messages = [{"role": "user", "content": "Hi"}]  # you write this...
# ...the API serializes it into the model's format FOR you. You don't see it.

# Open-source model you run yourself (LLaMA, Mistral):
prompt = "[INST] Hi [/INST]"  # YOU must format it correctly (or via a template)
```

✓ **Heart of this phase:** with hosted APIs, serialization is hidden. But the moment you run **open-source models** (for cost/control in AI work), *you* are responsible for formatting the prompt in the exact style that model expects.



Layer 3 — Expert (why this matters in practice)

1. Wrong format degrades output — silently

⚠ **Sneaky bug:** send a LLaMA-2 model a prompt without its `[INST]` tags and it still *runs* — just worse, because it misses the structural cues it was trained on. No error; the output is just subtly (or badly) worse. Matching the model's expected serialization is a real quality lever.

2. Chat templates — how it's handled in practice (Hugging Face)

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-chat-hf")

messages = [
    {"role": "user", "content": "Explain recursion."}
]

# the tokenizer applies the MODEL'S correct format automatically:
prompt = tokenizer.apply_chat_template(messages, tokenize=False)
# → "<s>[INST] Explain recursion. [/INST]" (LLaMA-2 format, done for you)
```

`apply_chat_template()` is the modern answer: you keep writing clean `messages`, and the tokenizer serializes them into *that specific model's* format. This is the bridge between the friendly structure and the raw string — learn this and you rarely format by hand.

3. Special tokens are real tokens

Markers like `<|im_start|>` or `[INST]` aren't just text — they're often **dedicated tokens** in the model's vocabulary, added during training. That's why they carry structural meaning: the model learned specific behavior around those exact tokens. Typing the characters manually vs. using the real special token can differ — another reason to use the model's official template rather than string-concatenating by hand.

4. Where you'll hit this in your work

- **Using open-source models** (LLaMA, Mistral via Hugging Face, Ollama, vLLM) — must match their format.
- **Fine-tuning** — training data must be serialized in the exact format the base model expects (get it wrong and fine-tuning fails). Why instruction-tuning formats like Alpaca exist.
- **Debugging bad open-source output** — often a serialization mismatch, not the model.
- **Switching models** — moving a prompt from GPT to LLaMA means re-serializing to the new format.

5. The mental model to carry forward

Your clean prompt (roles + content)
↓ serialization (the "style")
Raw text string with special tokens
↓
What the model actually reads

The rest of Phase 4 zooms into three styles — **Alpaca** (instruction-tuning text headers), **ChatML** (OpenAI's marker format), and **INST** (LLaMA-2's tags).

Interview Q&A Cards

Q: What is prompt serialization?

A: Converting a structured prompt (roles + content) into the exact plain-text string, with special tokens, that a given model was trained on. It's the bridge between the clean `messages` interface and what the model actually reads.

Q: Why do serialization formats differ between models?

A: Each model family was trained with its own markers and layout (Alpaca, ChatML, INST). The model recognizes structure from the exact format it saw in training, so matching it is required for best output.

Q: API vs open-source — who handles serialization?

A: Hosted APIs serialize your `messages` into the model's format behind the scenes. With open-source models you must format the raw string yourself — typically via a chat template.

Q: What happens if you use the wrong format?

A: Usually no error — the model still runs but produces subtly or badly worse output because it misses the structural cues it was trained on. That silent degradation makes format mismatches hard to spot.

🔗 **Memory hook:** *Serialization = your messages → the raw text string the model actually reads (with special tokens marking roles). APIs hide it; open-source models make it YOUR job. Wrong format = silently worse output. Use `apply_chat_template()` to do it right. Three styles coming: Alpaca, ChatML, INST.*